

Teman Tanyamu (dash_ai_me)

Lightweight self-hosted AI chat web app. An Open WebUI alternative with **under 100 MB RAM**. No Docker. No build step. One Node.js process + one SQLite file.

Stack	Node.js + Express 5 + better-sqlite3 + vanilla JS
AI Backend	9Router (<code>http://localhost:20128/v1</code> , OpenAI-compatible)
Providers	AG, Groq, BAI, OpenRouter, Nvidia, Cline — 497+ models
Frontend	<code>index.html</code> + <code>style.css</code> + <code>app.js</code> , zero framework, dark theme
Language	English (default) / Indonesian — runtime toggle
License	Private / personal use

Table of Contents

- 1. [Features](#)
- 2. [System Architecture](#)
- 3. [Request Flow Diagrams](#)
- 4. [Requirements](#)
- 5. [Installation](#)
- 6. [Management Script](#)
- 7. [Configuration](#)
- 8. [API Reference](#)
- 9. [Database Schema](#)
- 10. [Access from Other Devices](#)
- 11. [Performance](#)
- 12. [Security Notes](#)
- 13. [Troubleshooting](#)

Features

Chat

Feature	Description
Real-time streaming	SSE relay, word-by-word from 9Router

Feature	Description
Multi-session history	Per-user isolation in SQLite
Chat memory	Last 10 messages used as AI context
Rename / Delete chats	Via sidebar hover buttons
Chat search	Instant client-side filter by title (sidebar)
Markdown + code blocks	With streaming auto-close fence fix
Copy & Retry	Per-message actions
Stop generation	Abort mid-stream
Typing indicator	Animated dots while waiting
Temporary Chat	Zero history saved (endpoint: <code>/api/temp-chat</code>)

Users & Access

Feature	Description
Login	script per-user salt (<code>script1:salt:hash</code>), timing-safe compare; legacy static-salt hashes auto-upgrade on login
Session purge	Expired sessions deleted on every login
Roles	<code>admin</code> (full control) and <code>user</code> (chat only)
Daily quota	Per-user limit (default 50/day), admin unlimited
User management	Admin: add / delete / reset password / activate / deactivate
Instant session revoke	Deactivate or password reset kills sessions immediately
Quota reset	Admin can reset any user's daily usage

Models

Feature	Description
Auto-fetch	Model list pulled live from 9Router
Global default	Admin setting, applies to all users
Per-user override	Admin sets specific model per user
Per-chat model	Each chat remembers its model
Locked for users	Regular users cannot change model (server-enforced)

Appearance

Feature	Description
Dark theme	Open WebUI inspired palette
Chat wallpaper	6 presets + custom image upload (per-user, optional global); dark overlay only for image wallpapers, gradient presets render as-is
Custom avatar	Auto-downscaled upload, strict base64 validation
Sidebar minimize	Desktop: icon rail, state persisted · Mobile: closes the drawer
Bilingual UI	English / Indonesian, all strings translated
Responsive	Mobile drawer sidebar, 44px+ tap targets

System Architecture

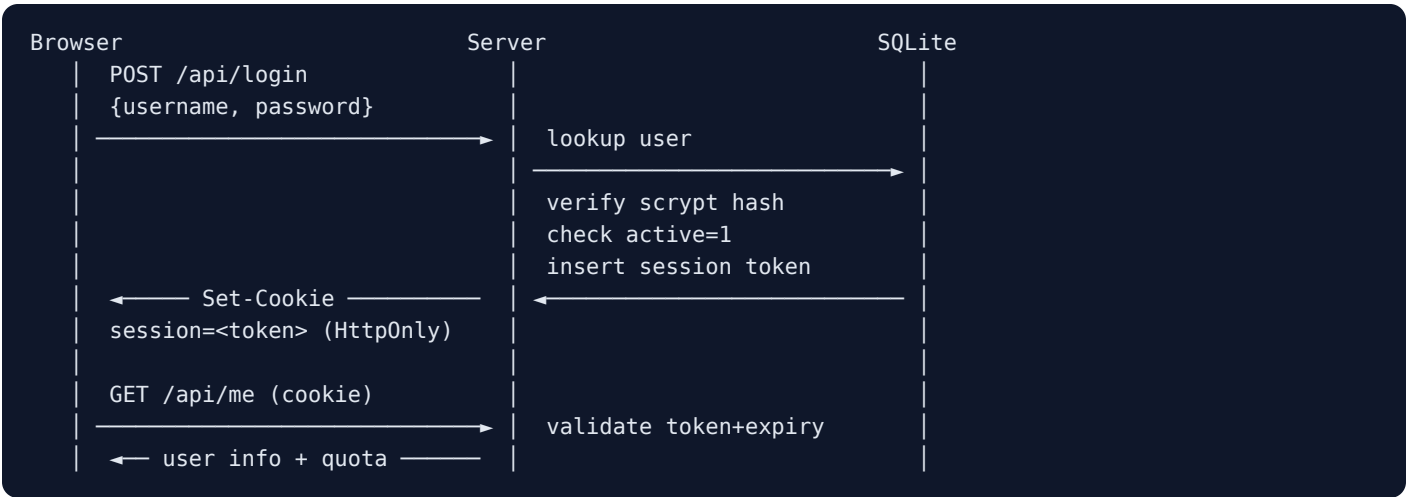


Key Design Decisions

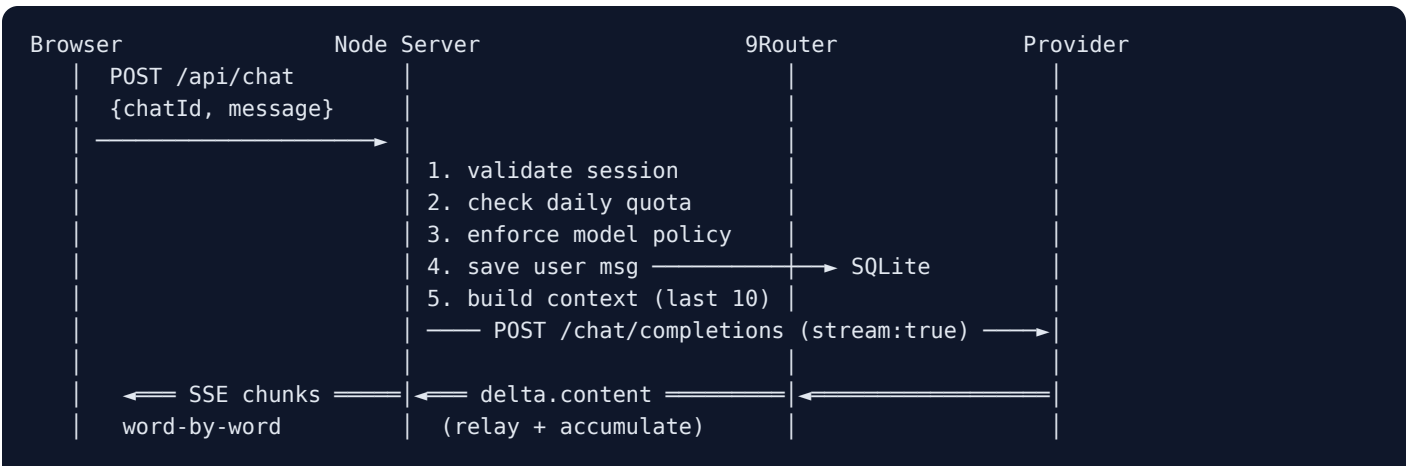
Decision	Reason
Single file backend (<code>server.js</code>)	Easy audit, zero complexity
SQLite over Postgres	Zero-config, WAL mode, single file backup
Server-side API key proxy	Key never reaches the browser
SSE relay (not direct)	Streaming works cross-device, CORS-free
Error codes (<code>ERR_*</code>)	Translated client-side per language
no-cache delivery	HTML: no-store · CSS/JS: no-cache (updates always fresh)
Split frontend (<code>style.css</code> + <code>app.js</code>)	Cacheable assets, lintable JS, CSP-ready

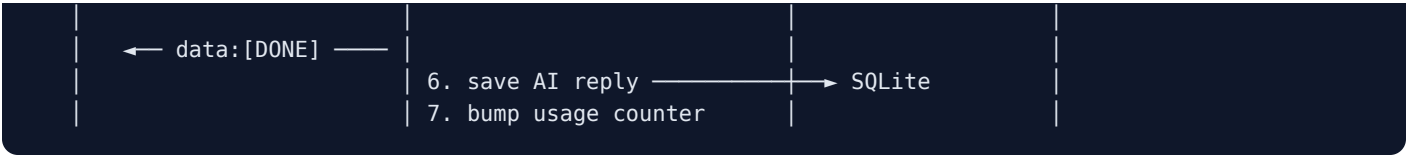
Request Flow Diagrams

1. Login Flow

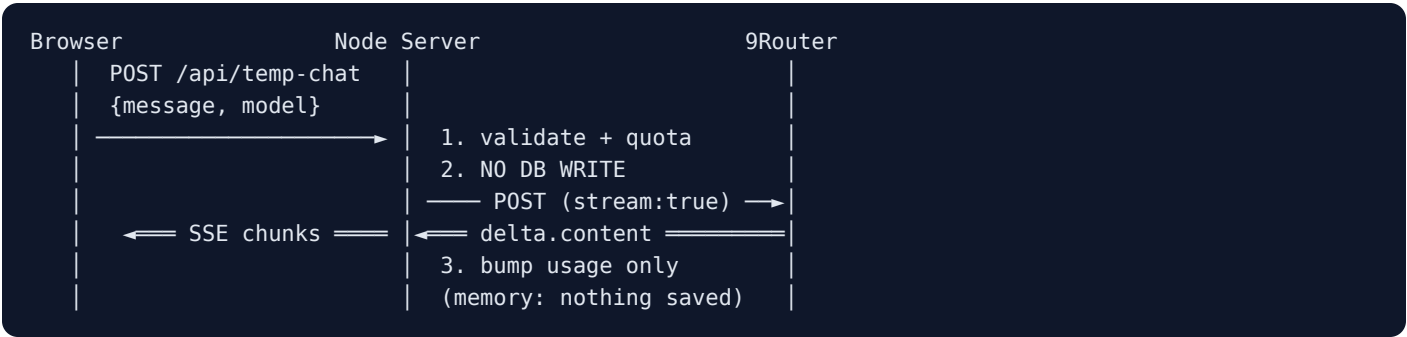


2. Streaming Chat Flow

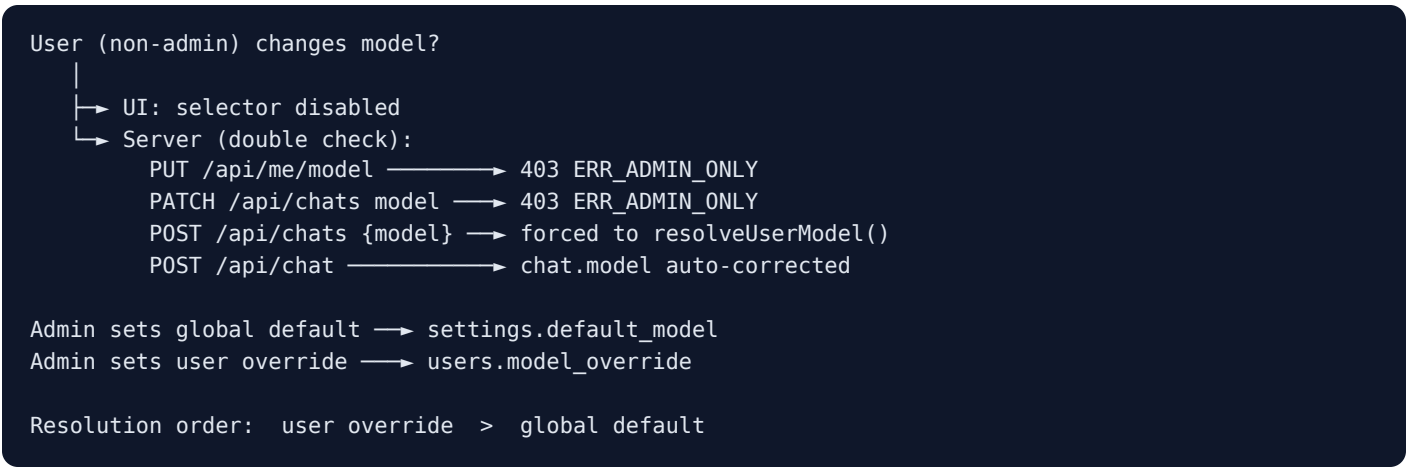




3. Temporary Chat Flow



4. Admin Model Policy



Requirements

Component	Minimum
OS	Linux (tested: Ubuntu 24.04)
Node.js	>= 20 (tested: v24.21)
9Router	Running, OpenAI-compatible, default port 20128
Disk	~50 MB (incl. node_modules)

Installation

```
cd dash_ai_me
npm install      # express + better-sqlite3
node server.js   # http://localhost:3000
```

First run creates the admin account automatically:

```
username: admin
password: admin123      # change immediately via UI
```

Management Script (manage.sh)

All-in-one service manager (systemd user service):

```
./manage.sh start      # Start service
./manage.sh stop       # Stop service
./manage.sh restart    # Restart service
./manage.sh status     # Status, port, RAM
./manage.sh port <1-65535> # Change port (persistent, auto-restart)
./manage.sh clear-cache # Clean SQLite WAL + npm cache + old journal
./manage.sh enable-boot # Auto-start at boot (enables linger)
./manage.sh disable-boot # Disable boot autostart
./manage.sh logs [n]   # Last n log lines (default 50)
./manage.sh password   # Reset admin password to admin123
./manage.sh help       # Show help
```

enable-boot runs `loginctl enable-linger` so the service runs without an active desktop session.

Configuration

Environment Variables

Variable	Default	Description
PORT	3000	Web server port
ROUTER_BASE	http://localhost:20128/v1	9Router API base URL
ROUTER_KEY	sk-a777...	9Router API key
ADMIN_PASSWORD	admin123	Initial admin password (first run only)

Example:

```
PORT=8080 ROUTER_BASE=http://192.168.1.10:20128/v1 node server.js
```

File Locations

Path	Description
chat.db	SQLite database (WAL mode)
public/index.html	Struktur halaman SPA (login + app shell)
public/style.css	Seluruh styling frontend
public/app.js	Seluruh logika frontend
server.js	Backend (single file)
.port	Persisted port number (created by <code>manage.sh port</code>)
<code>~/.config/systemd/user/dash-ai-me.service</code>	systemd unit

API Reference

All endpoints require the session cookie except `/api/login` and `/api/health`. Errors return `{"error": "ERR_*"}` codes, translated client-side.

Auth

Method	Endpoint	Body	Description
POST	<code>/api/login</code>	<code>{username, password}</code>	Sign in
POST	<code>/api/logout</code>	-	Destroy session
GET	<code>/api/me</code>	-	User info + quota + effective model
GET	<code>/api/health</code>	-	Liveness + DB check (no auth, for uptime monitors)

Account

Method	Endpoint	Body	Description
PUT	<code>/api/me/password</code>	<code>{current, next}</code>	Change own password
PUT	<code>/api/me/avatar</code>	<code>{avatar: dataURL null}</code>	Set/remove avatar
PUT	<code>/api/me/model</code>	<code>{model}</code>	Own model override (admin)
DELETE	<code>/api/me/model</code>	-	Clear override (admin)

Chats

Method	Endpoint	Body	Description
GET	/api/chats	-	List own chats
POST	/api/chats	{title?, model?}	Create chat
GET	/api/chats/:id	-	Chat + full messages
PATCH	/api/chats/:id	{title?, model?}	Rename / change model
DELETE	/api/chats/:id	-	Delete chat + messages

Chat & Preferences

Method	Endpoint	Body	Description
GET	/api/models	-	Model list from 9Router
POST	/api/chat	{chatId, message}	Streamed chat (SSE)
POST	/api/temp-chat	{message, model?}	Streamed temp chat (SSE, not saved)
GET	/api/wallpaper	-	Effective wallpaper
PUT	/api/wallpaper	{wallpaper\ null}	Own wallpaper

Admin

Method	Endpoint	Body	Description
GET	/api/admin/users	-	Users + today's usage
POST	/api/admin/users	{username, password, role, daily_quota}	Create user
DELETE	/api/admin/users/:id	-	Delete user
PUT	/api/admin/users/:id/password	{password}	Reset password (revokes sessions)
PUT	/api/admin/users/:id/active	{active: bool}	Activate / deactivate
PUT	/api/admin/users/:id/reset-quota	-	Reset daily usage
PUT	/api/admin/users/:id/model	{model\ null}	Per-user model override
GET	/api/admin/settings	-	Global settings
PUT	/api/admin/settings	{default_model}	Set global default model
PUT	/api/admin/wallpaper	{wallpaper\ null}	Set global wallpaper

Error Codes

Code	Meaning
ERR_UNAUTH	Not signed in
ERR_ADMIN_ONLY	Admin-only action
ERR_FIELDS_REQUIRED	Missing fields
ERR_LOGIN	Wrong credentials
ERR_INACTIVE	Account deactivated
ERR_PW_SHORT	Password under 4 chars
ERR_OLD_PW	Old password wrong
ERR_QUOTA	Daily quota exceeded
ERR_RATE_LIMITED	Too many failed logins: 5 attempts, then 2-minute IP block
ERR_NOT_FOUND	Resource not found
ERR_USERNAME_TAKEN	Username exists
ERR_TOO_BIG	Image too large
ERR_AVATAR_FORMAT	Avatar not an image
ERR_WALLPAPER_FORMAT	Wallpaper format invalid
ERR_SELF_DELETE	Cannot delete own account
ERR_SELF_DEACTIVATE	Cannot deactivate own account
ERR_BAD_BODY	Malformed JSON body

Database Schema

```

users      (id, username, password_hash, role, daily_quota,
            model_override, avatar, active, created_at)
            -- password_hash: "scrypt1:<salt>:<hash>" (per-user salt);
            -- legacy static-salt hex auto-upgrades on next login
sessions   (token, user_id, expires_at)                -- expired rows purged on login
chats      (id, user_id, title, model,
            created_at, updated_at)
messages   (id, chat_id, role, content, tokens, created_at)
usage      (user_id, date, request_count, tokens_used) -- PK (user_id, date)
settings   (key, value)
            -- keys: default_model, wallpaper, user:<id>:wallpaper

```

Access from Other Devices

Server binds to all interfaces. From phones / LAN devices:

```
http://<laptop-ip>:3000      e.g. http://20.20.100.6:3000
```

For access over the internet, a Cloudflare Tunnel works out of the box: `trust proxy` is enabled and `cf-connecting-ip` is honored for login rate limiting; the session cookie gets the `Secure` flag automatically when the request arrives via HTTPS.

Performance

Metric	Measured
RAM idle	20-30 MB
RAM peak (streaming)	50-65 MB
CPU idle	~0%
Boot time	< 3 s
Streaming first token	~0.5-2 s (provider dependent)

Security Notes

- Passwords: script with per-user random salt, stored as `script1:<salt>:<hash>`, verified with `crypto.timingSafeEqual`; legacy static-salt hashes upgrade automatically on the next successful login
- Sessions: HttpOnly cookie, SameSite=Lax, 7-day expiry; expired rows purged on login
- **Secure cookie flag**: auto-enabled when behind HTTPS (Cloudflare Tunnel sets `X-Forwarded-Proto`)
- **Login rate limiting**: 5 failed attempts per IP, then a 2-minute block (`ERR_RATE_LIMITED`)
- `trust proxy` enabled: client IP correctly detected behind Cloudflare (`CF-Connecting-IP`)
- **Security headers** on every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`
- **XSS hardening**: avatar & wallpaper validated server-side against strict formats (base64 image types / `linear-gradient(...)` / `#hex` only — no `svg+xml`, no attribute breakout); all user-supplied values escaped at render
- 9Router API key never sent to browser (server-side proxy only)
- Per-user chat isolation enforced in every SQL query
- Model policy enforced server-side (not just hidden in UI)
- Quota checked before every AI call; failed/aborted streams do not consume quota
- Deactivate / password reset revokes all sessions instantly
- Caching: HTML `no-store`, CSS/JS `no-cache` (updates always fresh)

Troubleshooting

Problem	Fix
"9Router offline"	Check: <code>curl http://localhost:20128/v1/models -H "Authorization: Bearer <key>"</code>
Forgot admin password	<code>./manage.sh password</code>
Port already in use	<code>./manage.sh port 3001</code>
Not auto-starting at boot	<code>./manage.sh enable-boot</code> , verify <code>loginctl show-user \$USER \ grep Linger</code>
Stale UI after update	Hard refresh: <code>Ctrl+Shift+R</code> (HTML/CSS/JS are all no-cache; rarely needed)
Full reset (nuclear)	<code>./manage.sh stop && rm -f chat.db* && ./manage.sh start</code> — all data gone